Energy-Aware Priority Scheduling in Smart Sensor Networks Using Policy-Guided Deep Reinforcement Learning

Anand Emmanuel Raju B.[1], Radhika N.[2], and Radhika G.[3]

Department of Computer Science and Engineering
Amrita School of Computing, Coimbatore
Amrita Vishwa Vidyapeetham, India
n_radhika@cb.amrita.edu, g_radhika@cb.amrita.edu,
cb.sc.p2cse25003@cb.students.amrita.edu

Abstract. This paper studies deep reinforcement learning (DRL) for packet scheduling in a dynamic wireless sensor network (WSN) carrying mixed high-priority (HP) and normal traffic. We couple ns-3 and OpenAI Gym through ns3-gym and train a prioritized-replay Double DQN scheduler with policy guidance that protects urgent HP service while penalizing wasteful behavior. Evaluation is performed with fixed seeds and repeated runs, using Strict Priority and Random as baselines. Across 20 matched evaluation runs that reuse the same environment seeds across policies, the learned policy strongly improves over Random and maintains high HP delivery, but remains below Strict Priority on throughput, packet-delivery ratio, HP delay, and HP delivery ratio. The results establish a clear performance boundary for this configuration and identify where objective redesign is required to improve HP-centric QoS. The paper contributes an artifact-backed pipeline, a priority-aware reward design with explicit HP-delivery instrumentation, and a statistically analyzed benchmark for future constrained and safety-aware DRL schedulers in WSNs.

Keywords: Smart Energy Systems · Wireless Sensor Networks · Reinforcement Learning · ns-3 · QoS · Priority Scheduling

1 Introduction

Battery-powered sensor nodes rarely get to choose their traffic load or their radio conditions, and this constrains almost every scheduling decision a wireless sensor network (WSN) has to make. When a node juggles bursty demand, a tight energy budget, and packets of unequal urgency, the resulting scheduling choice shapes packet delivery ratio (PDR), deadline compliance, and ultimately battery lifetime [4,7]. High-priority (HP) traffic is the least forgiving of the three: it has to arrive with low delay, and it has to keep doing so even as ordinary traffic grows around it.

Smart energy infrastructure makes this concrete. A sensing node that watches a feeder line or a substation needs to keep working for years with little power. At the time it has to send important messages about protection, faults and control quickly. It is a challenge to balance these two things. Staying working for a long time and sending urgent messages quickly. This problem is at the edge of communication engineering and intelligent computing. That is why we think of it as one problem, not two.

The Strict Priority scheduling method deals with this problem in a simple way. It always gives priority to messages first. This simplicity is why it is still widely used. However it struggles when things like traffic patterns or a nodes power start to change. Deep reinforcement learning is an alternative because it can adjust to new situations rather than following a fixed rule.. It has its own problems, like policies that change over time rewards that can be cheated and fairness that breaks down when there is a lot of random traffic [8,10].

Here we use a DQN scheduler that is guided by a policy. We run it through a simulation using ns-3. Ns3-gym. We ask a specific question: can a learned policy provide reliable service for important messages while staying close to a strong manually built baseline under conditions that are repeatable enough to trust? We are using the DQN scheduler and the ns-3 and ns3-gym simulation [5,2] to find the answer to this question, about the sensing node and its messages.

The main contributions are:

- A reproducible end-to-end pipeline for DRL-based WSN scheduling with ns-3 and ns3-gym, including artifact export for run-level verification.
- A priority-aware reward and validation design that explicitly tracks HP generation, HP delivery, and HP delay.
- A run-level statistical comparison against Strict Priority and Random over 20 matched evaluation runs.
- An empirical study of the policy trade-off: normal traffic latency and bigger improvements compared to Random but not as good HP-focused QoS, as Strict Priority.

2 Related Work

A number of survey papers now trace how RL and DRL have moved into communication systems more broadly – scheduling, routing, medium access control – and the consistent message is that the potential is real but so are the obstacles: these methods are sample-hungry, hard to certify as safe, and prone to falling apart once the traffic regime they were trained on shifts [8,4].

Within WSNs specifically the research is divided into scheduling, routing and energy management. On the scheduling side Leong *et al.* Show that a DRL controller can track uncertainty in a cyber-physical sensing setting well enough to make effective transmission decisions. WSNs are important. WSNs have applications. WSNs are an area of study. [7]. Sharma and Chauhan focus, on coverage and connectivity instead. They keep nodes without wasting energy. They use a distributed RL scheme. They focus on coverage and connectivity. They keep

nodes without wasting energy. They use a distributed RL scheme. [11]. Dutta and Biswas focus on the medium-access layer. They use distributed reinforcement learning. This helps keep channel access. It is important as IoT and sensor deployments grow [3]. Barker and Swany study routing. They look at cooperative reinforcement learning agents. These agents work together to choose paths across the network [1].

A second group of research is more similar to the way we created our objective. Mishra and Liang use reinforcement learning to make sleep and wake decisions in order to save energy without affecting how well tasks get done [9]. Nagib *et al.* and his team describe rules for making deep reinforcement learning safer and quicker to train in wireless systems [10], and Kim *et al.* and others use reinforcement learning inside a time-sensitive networking system to adjust how traffic is managed. This situation is similar to our focus, on meeting deadlines [6].

What sets this paper apart from the work above is less the learning algorithm itself – Double DQN is well established – and more the instrumentation built around it: HP generation and delivery are tracked explicitly rather than folded into an aggregate reward, model selection is gated on a validation score rather than the training curve, and the run-level artifacts needed to check every number in Section 5 independently are released alongside the paper.

3 System Model and Problem Formulation

3.1 Network and Traffic Model

We simulate 100 sensor nodes and one sink, with queue-aware state and dynamic network conditions. The control interval is 100 ms. Each episode spans 5000 decision steps. Traffic contains two service classes: HP and normal. Per-class queueing and timeout behavior are tracked by the simulator and exported as end-of-episode metrics.

3.2 MDP Definition

At each decision step, the scheduler observes a 6-dimensional normalized state vector

$$\mathbf{s}_t = [q_t^N,\ q_t^H,\ e_t,\ a_t^N,\ a_t^H,\ d_t], \tag{1}$$

where q_t^N and q_t^H are normal/HP queue occupancy proxies, e_t is residual-energy proxy, a_t^N and a_t^H represent normalized oldest-packet ages, and d_t is a neighborhood-density proxy.

The action space is discrete:

$$\mathcal{A} = \{\text{send HP},\ \text{send normal},\ \text{drop normal},\ \text{sleep},\ \text{drop HP}\}. \tag{2}$$

3.3 Reward Design

The reward combines delivery gains, penalties for drops/timeouts, delay pressure, energy usage, and policy guidance. Let r_t^{base} denote the weighted sum of transmission and penalty terms and r_t^{guide} the HP-guidance term. The total reward is

$$r_t = \tanh\left(\frac{r_t^{\text{base}} + r_t^{\text{guide}}}{100}\right). \tag{3}$$

Policy guidance is defined as

$$r_t^{\text{guide}} = \begin{cases} +\alpha, & a_t = \text{send HP and } q_t^H > 0 \\ -\beta, & a_t \neq \text{send HP},\ q_t^H > 0,\ a_t^H > \tau_u \\ 0, & \text{otherwise} \end{cases} \tag{4}$$

with $\alpha = 6.0$, $\beta = 8.0$, and urgency threshold $\tau_u = 0.55$. This soft guidance avoids hard-coded imitation of Strict Priority while discouraging neglect of urgent HP packets.

4 Methodology

4.1 Learning Agent

The learning agent uses a Double DQN-style target construction with prioritized replay, soft target-network updates, gradient clipping, and cosine learning-rate scheduling. Replay capacity is 100k transitions and warm-up is 4k steps. The design aligns with common DRL-for-networking patterns reported in recent surveys [8,4].

Training uses ϵ-greedy exploration with multiplicative decay to a floor of 0.05. For evaluation, ϵ is set to zero. This separation reduces evaluation noise caused by exploration artifacts.

4.2 Baselines

- **Strict Priority**: serves HP whenever available, else normal, else sleep.
- **Random**: uniform random action selection.

4.3 Training and Evaluation Protocol

Training and evaluation are seed-controlled. The model is selected by validation score over five fixed validation seeds. Final reporting uses 20 matched evaluation runs per policy.

The same 20 environment seeds are reused across all policies to keep evaluation conditions matched run-by-run.

The validation score is made up of a things: PDR, PLR, HP delivery ratio and HP delay. These are all combined into a number that is not too big and

Table 1. Simulation and training configuration.

Category	Value
Simulator	ns-3.40 with ns3-gym interface
Nodes / sink	100 nodes + 1 sink
Control interval	100 ms
Episode length	5000 steps
Training epochs	50 (early stopping patience: 15)
Evaluation runs	20 per policy
State (or) action size	6 / 5
Replay-buffer	100,000 (prioritized replay)
Mini batch size	128
Warm up steps	4000
Discount factor (γ)	0.99
Optimizer	Adam (1×10^{-4} initial LR)
Target update	Soft update ($\tau = 0.005$)
Exploration	ϵ: 1.0 $\rightarrow$ 0.05, decay 0.99995
Random seeds	Global base seed + fixed validation seeds

not too small. If HP packets are made but none of them get delivered that is a problem. It gets a penalty. This is to fix a problem that was seen before where the old way of scoring could make it seem like everything is fine when really the HP is not getting what it needs.

4.4 Experimental Setup

Table 1 gives us the details, about the simulator and the training setup that was used to get the results that we are talking about.

4.5 Metrics and Statistical Testing

Primary metrics are throughput, PDR, PLR, average normal delay, average HP delay, energy per delivered bit, and HP delivery ratio. We report mean $\pm$ 95% confidence interval (CI).

For pairwise comparison (DQN vs. Strict Priority), runs are matched by seed. Normality is assessed on paired differences using the Shapiro–Wilk test at $\alpha = 0.05$. We use a paired t-test when the paired differences pass normality checks; otherwise we use the Wilcoxon signed-rank test. Effect size is reported as paired Cohen's d_z, computed from the 20 run-level paired differences.

Implementation uses a Python control loop connected to ns-3 via ns3-gym. Seeds are fixed across Python, NumPy, and PyTorch for run reproducibility. All reported summary values are generated from exported run-level artifacts (‘raw$_r esults.csv$‘, ‘$training_l og.csv$‘, ‘$final_r esults.xlsx$‘).

5 Results

Table 2 shows that the proposed policy-guided DQN clearly improves over Random across all key metrics, but does not exceed Strict Priority in this workload.

Table 2. Final performance over 20 evaluation runs (mean ± 95% CI). Higher is better for throughput, PDR, and HP delivery ratio; lower is better for PLR, delays, and energy.

Policy	Throughput (kbps)	PDR (%)	PLR (%)	Avg. Normal Delay (s)	Avg. HP Delay (s)	Energy (nJ/bit)	HP Delivery Ratio (%)
Policy-Guided DQN	7.0267 ± 0.0702	50.3291 ± 0.6022	40.3309 ± 0.8373	0.4287 ± 0.0261	0.1431 ± 0.0026	443.8708 ± 16.9422	95.7779 ± 0.2916
Strict Priority	7.9104 ± 0.0054	56.6625 ± 0.3137	34.7224 ± 0.5326	0.6171 ± 0.0384	0.0274 ± 0.0015	417.0370 ± 17.2341	98.3374 ± 0.1941
Random	2.8000 ± 0.0350	20.0709 ± 0.3314	71.4943 ± 0.5450	0.8545 ± 0.0597	0.3521 ± 0.0129	806.7023 ± 22.7489	34.4862 ± 0.8563

Table 3. Paired run-level statistical comparison (n=20 matched runs): Policy-Guided DQN vs. Strict Priority.

Metric	Test	p-value	Paired Cohen's d_z
Throughput (kbps)	Paired t-test	2.656×10^{-16}	−5.8046
PDR (%)	Paired t-test	1.655×10^{-15}	−5.2568
PLR (%)	Paired t-test	4.749×10^{-13}	3.8504
Avg. Normal Delay (s)	Wilcoxon signed-rank	1.907×10^{-6}	−3.3284
Avg. HP Delay (s)	Paired t-test	3.038×10^{-26}	19.6001
Energy (nJ/bit)	Paired t-test	1.687×10^{-14}	4.6310
HP Delivery Ratio (%)	Paired t-test	2.176×10^{-11}	−3.1005

Against Random, policy-guided DQN improves throughput (7.0267 vs. 2.8000 kbps), PDR (50.33% vs. 20.07%), HP delay (0.1431 vs. 0.3521 s), and energy efficiency (443.87 vs. 806.70 nJ/bit). Against Strict Priority, however, DQN is behind on throughput, PDR, PLR, HP delay, and HP delivery ratio. A positive trade-off is that DQN yields lower normal-packet delay than Strict Priority.

Statistical tests in Table 3 indicate that most DQN-vs-Strict differences are significant, with large effect sizes for throughput, PDR, and HP-delay-related outcomes.

5.1 Training Dynamics and Policy Behavior

5.2 Comparative Views

5.3 Interpretation

Figure 1 indicates stable training progression and a convergent validation trend, suggesting the policy did not diverge.

Figures 2 and 3 show the same pattern as the numeric tables: DQN is consistently above Random and below Strict Priority for HP-centric service quality. The learned controller remains stable and clearly non-random, but does not yet achieve strict-priority-level HP delay.

5.4 Trade-Off Interpretation

The central finding is straightforward: the learned policy is clearly stronger than Random, but does not outperform Strict Priority on HP-centric QoS in this scenario. The interesting behavior is in the middle ground: DQN maintains high HP delivery while reducing normal-packet delay relative to Strict Priority. This suggests that the learned controller is balancing objectives rather than collapsing into random behavior.

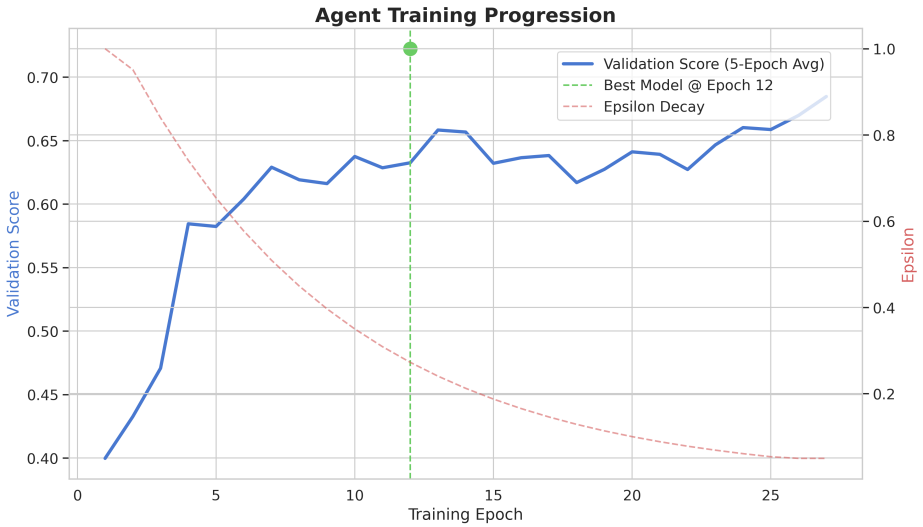

Fig. 1. Training progression and validation-score trend.

5.5 Error Pattern Analysis

The observed performance gap is consistent with objective misalignment under competition between urgency and efficiency terms. DQN reduces normal delay and avoids the pathological no-HP-delivery behavior observed in earlier development stages, but this comes with insufficient aggressiveness on ultra-low HP-delay optimization when compared with Strict Priority.

Another practical observation is that reward improvements in training do not always map linearly to deployment-facing metrics. This reinforces the need for metric-level model selection and post-training stress testing rather than relying on reward curves alone.

For deployment-oriented WSN systems, these results suggest a pragmatic policy choice. If the operational requirement is strict minimization of HP delay, Strict Priority remains the safer default in this scenario. If adaptive balancing between classes and energy is required, DRL remains promising but likely needs explicit safety constraints or hybrid fallback rules.

6 Discussion

The DRL policy is viable and stable under policy guidance, and HP starvation is no longer hidden because HP generation/delivery are explicitly tracked. Still, baseline superiority suggests that one or more factors constrain performance: (i) limited observability in the compact 6D state, (ii) reward-weight mismatch between short-term delay control and long-term throughput, or (iii) policy-class limitations under highly variable traffic.

Fig. 2. Policy comparison using means and confidence intervals across key metrics.

Future work should prioritize constrained/safe RL formulations, richer function approximation (e.g., distributional or dueling variants), and stronger generalization protocols (domain randomization and out-of-distribution traffic tests) [10,6]. Adding stronger non-RL adaptive baselines (e.g., weighted-priority or deadline-aware heuristics) is also necessary for broader comparative validity.

7 Threats to Validity

Results are tied to the selected simulator configuration, traffic intensity, timeout settings, and channel assumptions. Although seeds are controlled and repeated, conclusions can change under different workloads or hardware constraints. The study is simulation-based; direct deployment claims are out of scope.

Another validity risk is objective selection: emphasizing HP service can reduce normal-service aggressiveness. Simulator-to-reality transfer is also a known gap, and broader baseline coverage is still needed beyond Strict Priority and Random.

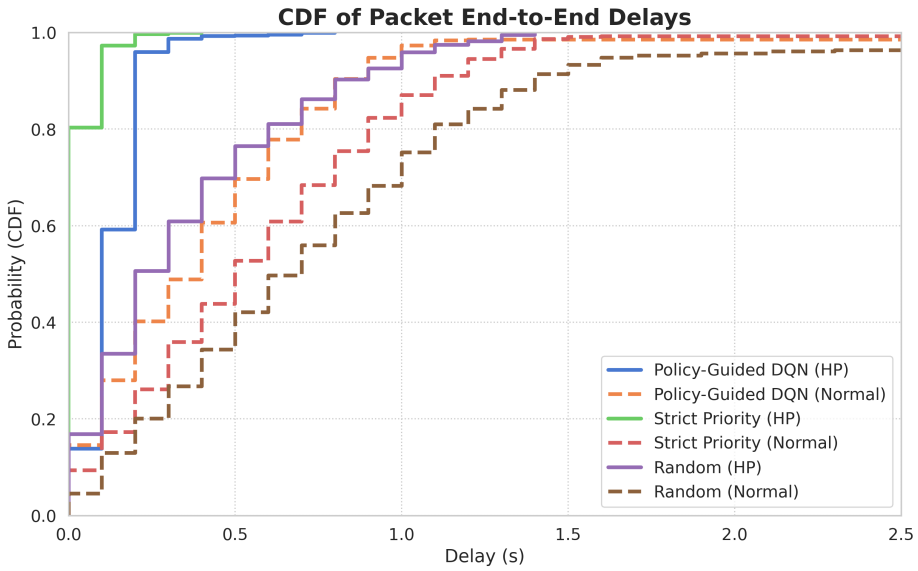

Fig. 3. Delay CDF comparison for normal and HP traffic.

8 Conclusion

This study presents an end-to-end reproducible DRL scheduling pipeline for dynamic WSNs using ns-3 and ns3-gym. The proposed policy significantly outperforms Random and maintains high HP service, but does not surpass Strict Priority in the current scenario. The value of this work is a transparent benchmark with full artifacts, explicit HP-service instrumentation, and statistically grounded analysis that can guide next-generation constrained/safe DRL schedulers.

In summary, the contribution is methodological and empirical: we provide a reliable benchmark, identify concrete failure/gap patterns, and establish a reproducible foundation for stronger constrained RL designs targeted at smart-energy communication networks.

Reproducibility Package

The submission package includes the exact result CSV, training log, summary workbook, and the figures/tables used in this manuscript.

References

1. Barker, A., Swany, M.: Distributed cooperative reinforcement learning for wireless sensor network routing. In: 2022 IEEE Wireless Communications and Net-

working Conference (WCNC). pp. 2565–2570 (2022). https://doi.org/10.1109/WCNC51071.2022.9771591
2. Boni, A.K.C.S., Hassan, H., Drira, K.: Task offloading in autonomous iot systems using deep reinforcement learning and ns3-gym. In: Proceedings of the 11th International Conference on the Internet of Things. pp. 17–24 (2021). https://doi.org/10.1145/3494322.3494325
3. Dutta, H., Biswas, S.: Distributed reinforcement learning for scalable wireless medium access in iots and sensor networks. Computer Networks **202**, 108662 (2022). https://doi.org/10.1016/j.comnet.2021.108662
4. Frikha, M.S., Gammar, S.M., Lahmadi, A., Andrey, L.: Reinforcement and deep reinforcement learning for wireless internet of things: A survey. Computer Communications **178**, 98–113 (2021). https://doi.org/10.1016/j.comcom.2021.07.014
5. Gawlowicz, P., Zubow, A.: ns-3 meets openai gym: The playground for machine learning in networking research. In: Proceedings of the ACM International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems (MSWiM) (2019)
6. Kim, H., Kim, Y.J., Kim, W.T.: Deep reinforcement learning-based adaptive scheduling for wireless time-sensitive networking. Sensors **24**(16), 5281 (2024). https://doi.org/10.3390/s24165281
7. Leong, A.S., Ramaswamy, A.G., Quevedo, D.E., Karl, H., Shi, L.: Deep reinforcement learning for wireless sensor scheduling in cyber–physical systems. Automatica **113**, 108759 (2020). https://doi.org/10.1016/j.automatica.2019.108759
8. Luong, N.C., Hoang, D.T., Gong, S., Niyato, D., Wang, P.p., Liang, Y.C., Kim, D.I.: Applications of deep reinforcement learning in communications and networking: A survey. IEEE Communications Surveys & Tutorials **21**(4), 3133–3174 (2019). https://doi.org/10.1109/COMST.2019.2916583
9. Mishra, S., Liang, J.M.: Dynamic sleep scheduling for wireless sensor transmissions based on reinforcement learning. IEEE Sensors Letters **7**(11), 1–4 (2023). https://doi.org/10.1109/LSENS.2023.3327002
10. Nagib, A.M., Abou-Zeid, H., Hassanein, H.S.: Toward safe and accelerated deep reinforcement learning for next-generation wireless networks. IEEE Network **37**(2), 182–189 (2023). https://doi.org/10.1109/MNET.106.2100578
11. Sharma, A., Chauhan, S.: A distributed reinforcement learning based sensor node scheduling algorithm for coverage and connectivity maintenance in wireless sensor network. Wireless Networks **26**(6), 4411–4429 (2020). https://doi.org/10.1007/s11276-020-02350-y